CHAPTER EIGHT
BASIC STRUCTURAL MODELING

Class

- A class is a description of a set of objects that share the same attributes, operations, relationships, and semantics.
- A class implements one or more interfaces.
- The UML provides a graphical representation of class

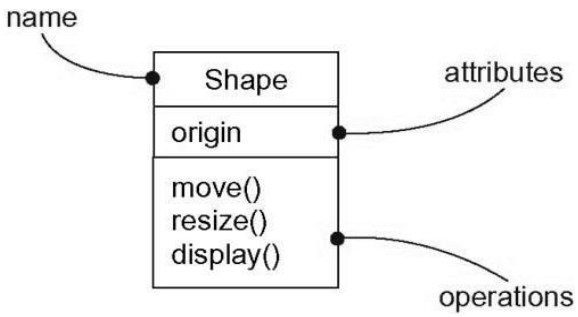


Fig: Graphical Representation of Class in UML

Terms and Concepts

Names

- Every class must have a name that distinguishes it from other classes.
- A name is a textual string that name alone is known as a simple name.
- A path name is the class name prefixed by the name of the package in which that class lives.

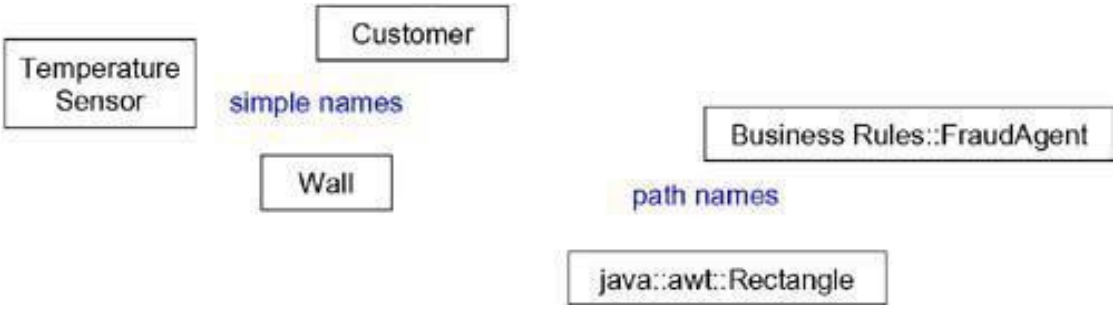


Fig: Simple and Path Names

Attributes

- An attribute is a named property of a class that describes a range of values that instances of the property may hold.
- A class may have any number of attributes or no attributes at all.
- An attribute represents some property of thing you are modeling that is shared by all objects of that class
- You can further specify an attribute by stating its class and possibly a default initial value.

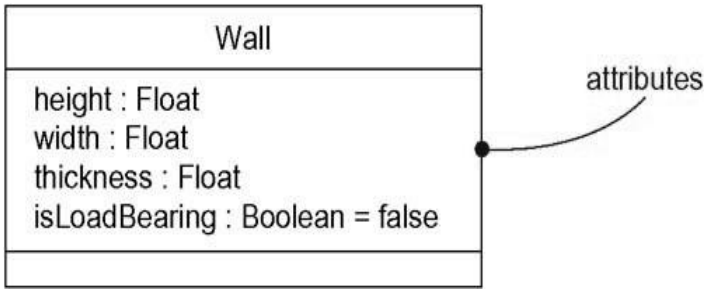


Fig: Attributes and Their Class

Operations

- An operation is the implementation of a service that can be requested from any object of the class to affect behavior.
- A class may have any number of operations or no operations at all
- Graphically, operations are listed in a compartment just below the class attributes
- You can specify an operation by stating its signature, covering the name, type, and default value of all parameters and a return type.

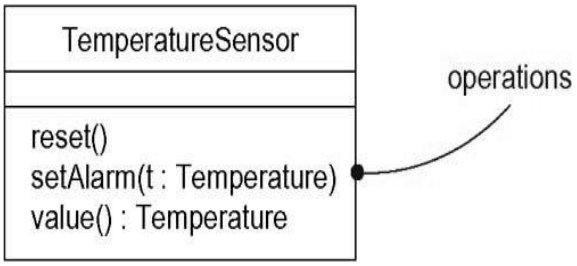
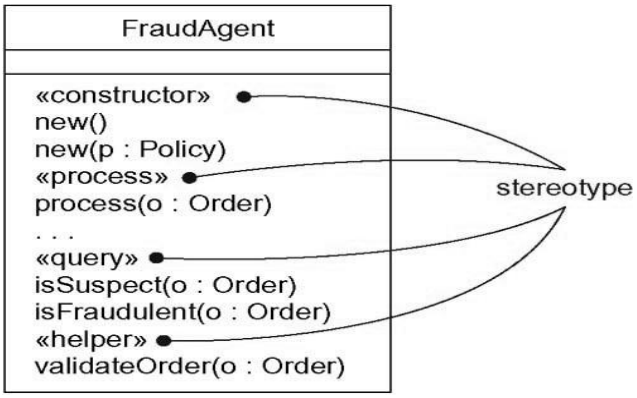


Fig: Operations

Organizing Attributes and Operations

To better organize long lists of attributes and operations, you can also prefix each group with a descriptive category by using stereotypes



Responsibilities

- A Responsibility is a contract or an obligation of a class
- When you model classes, a good starting point is to specify the responsibilities of the things in your vocabulary.
- A class may have any number of responsibilities, although, in practice, every well-structured class has at least one responsibility and at most just a handful.
- Graphically, responsibilities can be drawn in a separate compartment at the bottom of the class icon

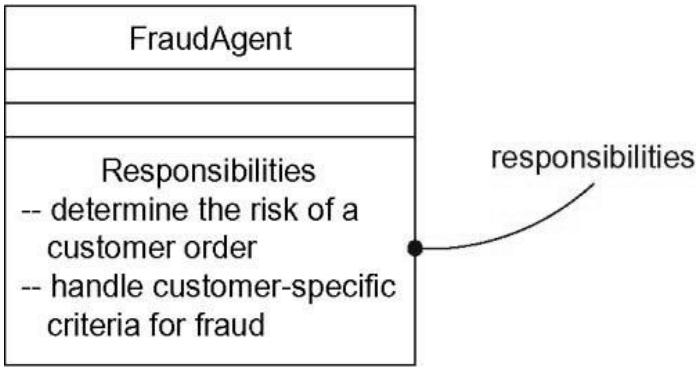


Fig: Responsibilities

Common Modeling Techniques

Modeling the Vocabulary of a System

- You'll use classes most commonly to model abstractions that are drawn from the problem you are trying to solve or from the technology you are using to implement a solution to that problem.
- They represent the things that are important to users and to implementers

To model the vocabulary of a system

- Identify those things that users or implementers use to describe the problem or solution.
- Use CRC cards and use case-based analysis to help find these abstractions.
- For each abstraction, identify a set of responsibilities.
- Provide the attributes and operations that are needed to carry out these responsibilities for each class.

Figure shows a set of classes drawn from a **retail system**, including Customer, Order, and Product. This figure includes a few other related abstractions drawn from the vocabulary of the problem, such as Shipment (used to track orders), Invoice (used to bill orders), and Warehouse (where products are located prior to shipment). There is also one solution-related abstraction, Transaction, which applies to orders and shipments.

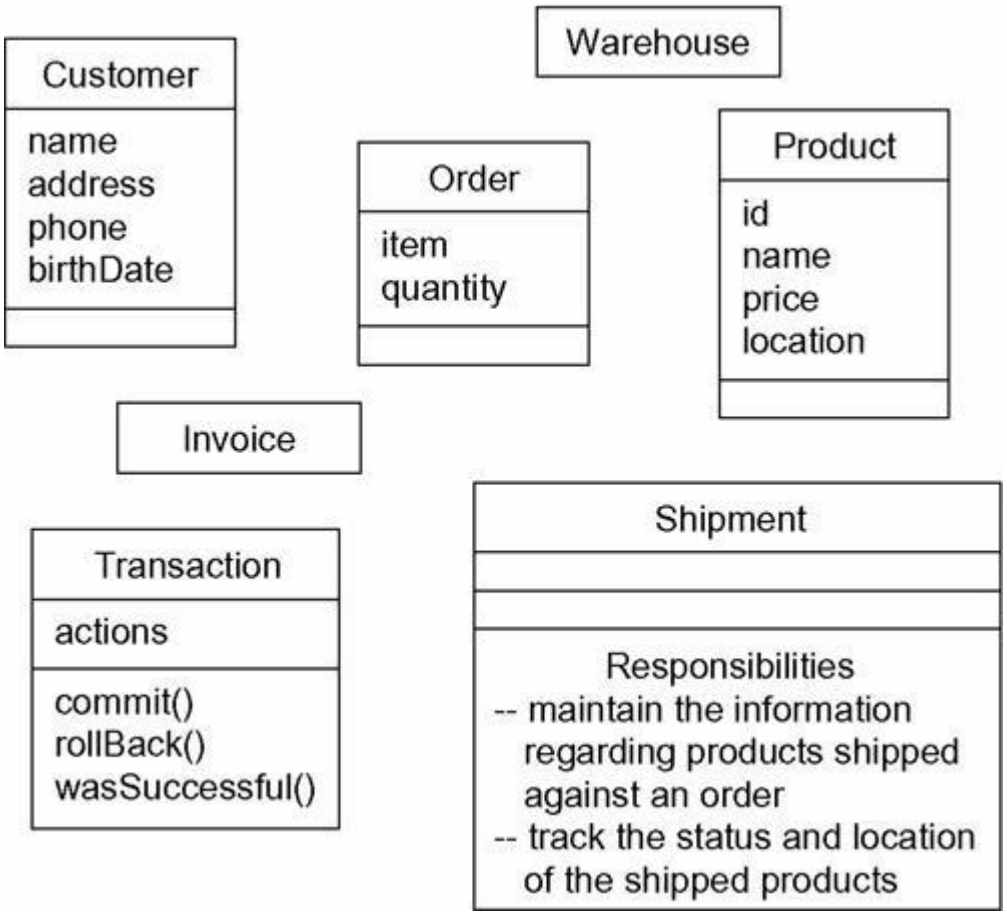


Fig: Modeling the Vocabulary of a System

Modeling the Distribution of Responsibilities in a System

- Once you start modeling more than just a handful of classes, you will want to be sure that your abstractions provide a balanced set of responsibilities.

To model the distribution of responsibilities in a system

- Identify a set of classes that work together closely to carry out some behavior.
- Identify a set of responsibilities for each of these classes.
- Look at this set of classes as a whole, split classes that have too many responsibilities into smaller abstractions, collapse tiny classes that have trivial responsibilities into larger ones, and reallocate responsibilities so that each abstraction reasonably stands on its own.
- Consider the ways in which those classes collaborate with one another, and redistribute their responsibilities accordingly so that no class within a collaboration does too much or too little.

For example, figure shows a set of classes drawn from Smalltalk, showing the distribution of responsibilities among Model, View, and Controller classes. Notice how all these classes work together such that no one class does too much or too little.

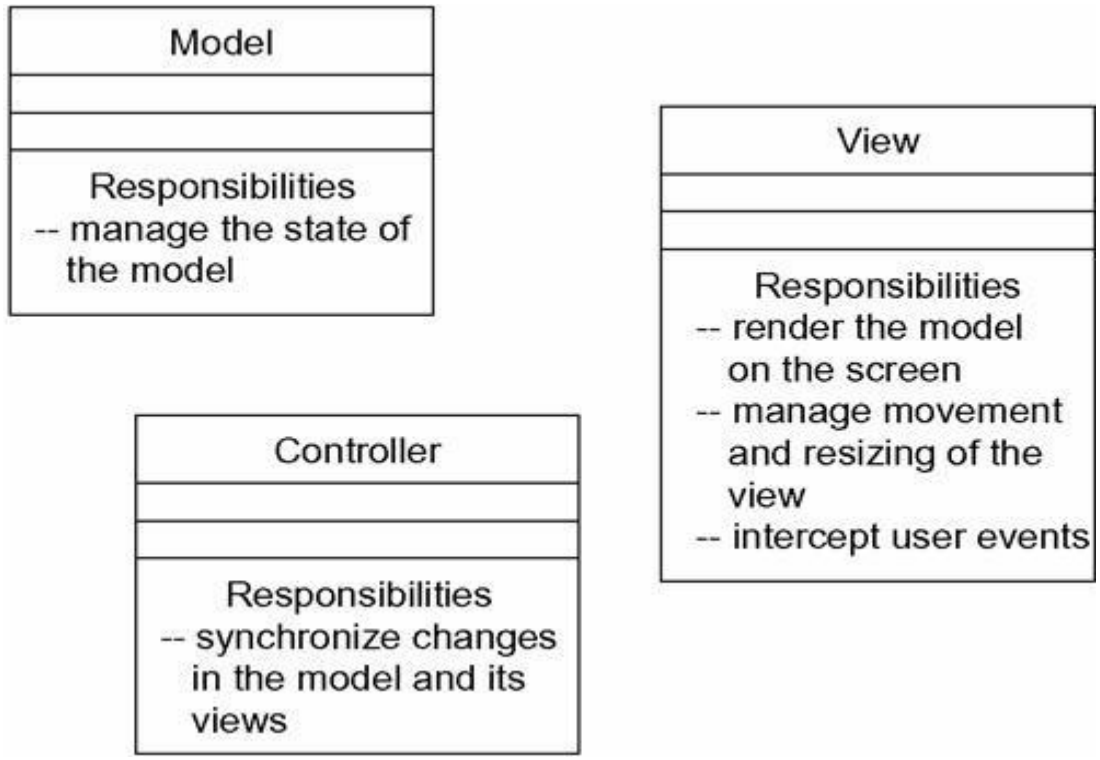


Fig: Modeling the Distribution of Responsibilities in a System

Modeling Nonsoftware Things

- Sometimes, the things you model may never have an analog in software
- Your application might not have any software that represents them.

To model nonsoftware things

- Model the thing you are abstracting as a class.
- If you want to distinguish these things from the UML's defined building blocks, create a new
- Building block by using stereotypes to specify these new semantics and to give a distinctive visual cue.
- If the thing you are modeling is some kind of hardware that itself contains software, consider
- Modeling it as a kind of node, as well, so that you can further expand on its structure.

Figure below shows, it's perfectly normal to abstract humans (like AccountsReceivableAgent) and hardware (like Robot) as classes, because each represents a set of objects with a common structure and a common behavior.

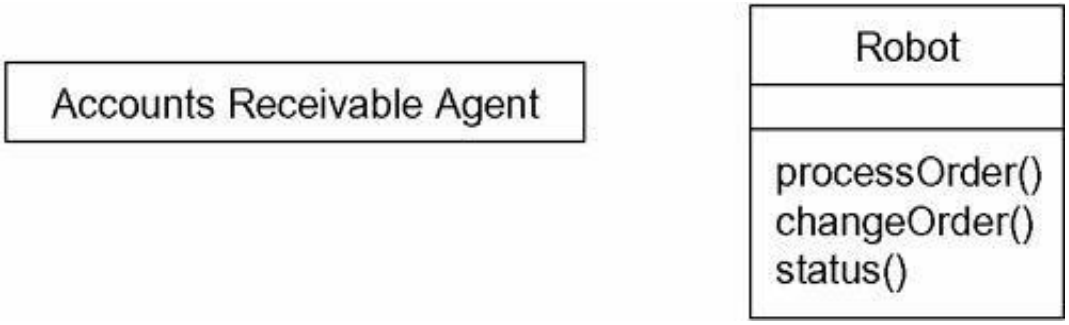


Fig: Modeling Nonsoftware Things

Modeling Primitive Types

- At the other extreme, the things you model may be drawn directly from the programming language you are using to implement a solution.
- Typically, these abstractions involve primitive types, such as integers, characters, strings, and even enumeration types.

To model primitive types

- Model the thing you are abstracting as a type or an enumeration, which is rendered using class notation with the appropriate stereotype.
- If you need to specify the range of values associated with this type, use constraints.

Figure shows, these things can be modeled in the UML as types or enumerations, which are rendered just like classes but are explicitly marked via stereotypes. Things like integers (represented by the class Int) are modeled as types, and you can explicitly indicate the range of values these things can take on by using a constraint. Similarly, enumeration types, such as Boolean and Status, can be modeled as enumerations, with their individual values provided as attributes.

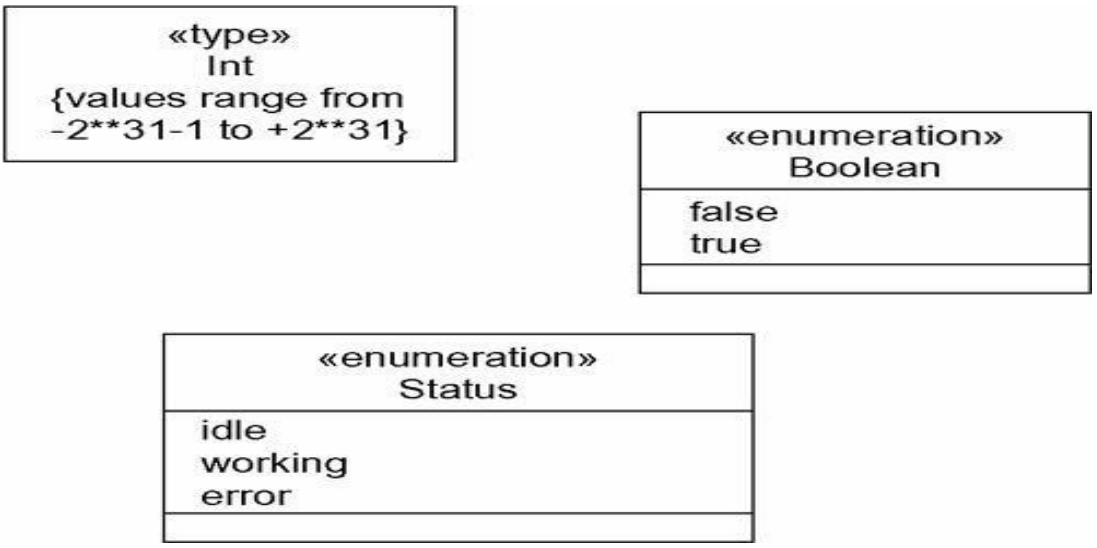


Fig: Modeling Primitive Types

Relationships

- In the UML, the ways that things can connect to one another, either logically or physically, are modeled as relationships.
- Graphically, a relationship is rendered as a path, with different kinds of lines used to distinguish the kinds of relationships
- In object-oriented modeling, there are three kinds of relationships that are most important:

  Dependencies
  Generalizations
  Associations

Dependencies

- Dependencies are using relationships. For example, pipes depend on the water heater to heat the water they carry..

Generalizations

- Generalizations connect generalized classes to more-specialized ones in what is known as subclass/superclass or child/parent relationships.
- For example, a bay window is a kind of window with large, fixed panes; a patio window is a kind of window with panes that open side to side.

Associations

- Associations are structural relationships among instances.
- For example, rooms consist of walls and other things; walls themselves may have embedded doors and windows; pipes may pass through walls..

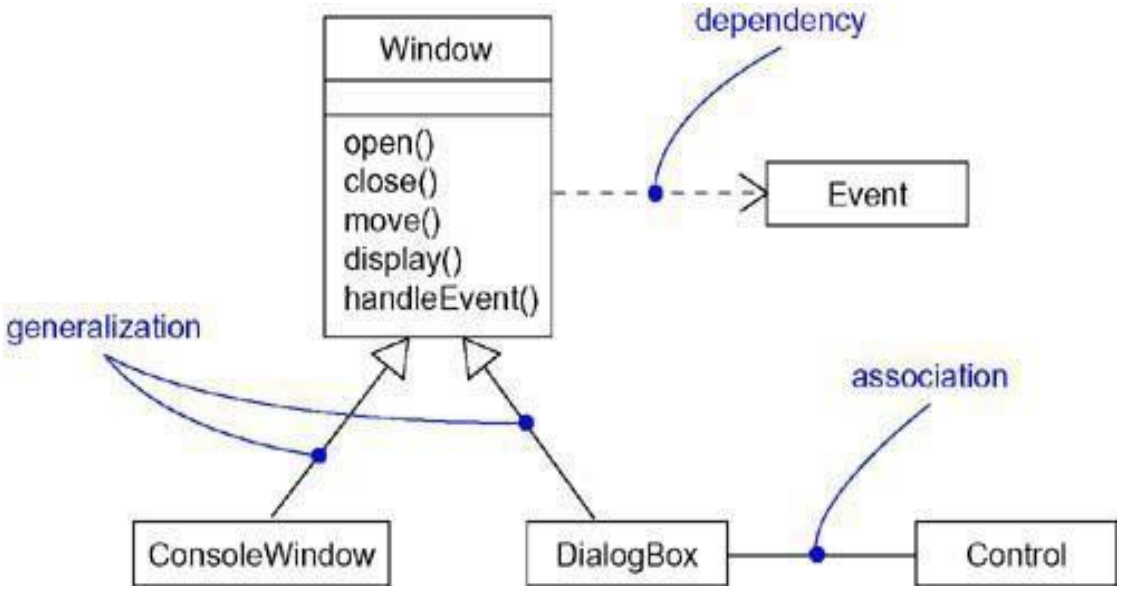


Fig: Relationships

Terms and Concepts

- A relationship is a connection among things. In object-oriented modeling, the three most important relationships are dependencies, generalizations, and associations.
- Graphically, a relationship is rendered as a path, with different kinds of lines used to distinguish the kinds of relationships.

Dependency

- A dependency is a using relationship that states that a change in specification of one thing may affect another thing that uses it but not necessarily the reverse.
- Graphically dependency is rendered as a dashed directed line, directed to the thing being depended on.
- Most often, you will use dependencies in the context of classes to show that one class uses another class as an argument in the signature of an operation

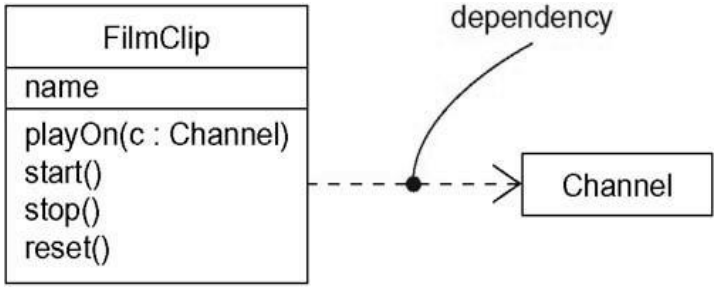


Fig: Dependencies

Generalization

- A generalization is a relationship between a general thing (called the super class or parent)and a more specific kind of that thing (called the subclass or child).
- Generalization means that the child is substitutable for the parent. A child inherits the properties of its parents, especially their attributes and operations
- An operation of a child that has the same signature as an operation in a parent overrides the operation of the parent; this is known as polymorphism.
- Graphically generalization is rendered as a solid directed line with a large open arrowhead, pointing to the parent

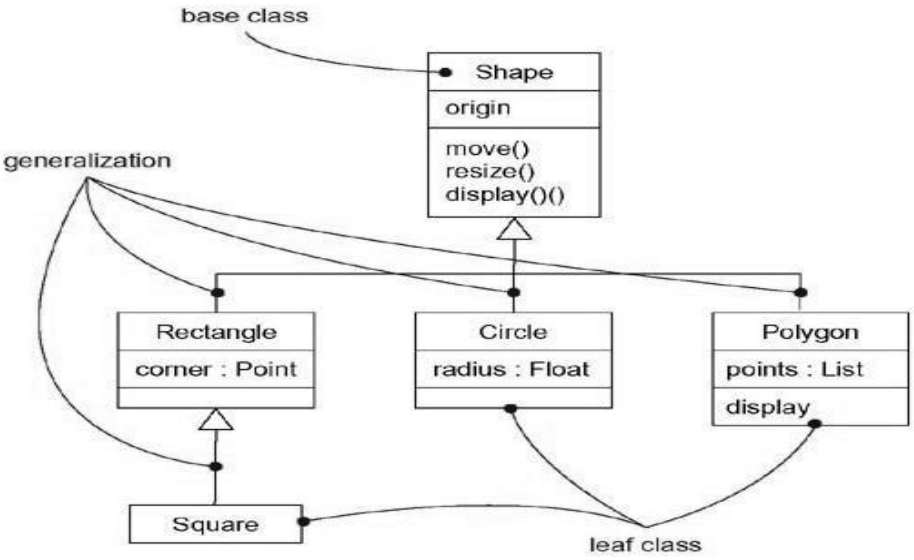


Fig: Generalization

Association

- An association is a structural relationship that specifies that objects of one thing are connected to objects of another
- An association that connects exactly two classes is called a binary association
- An associations that connect more than two classes; these are called n-ary associations.
- Graphically, an association is rendered as a solid line connecting the same or different classes.
- Beyond this basic form, there are four adornments that apply to associations

Name

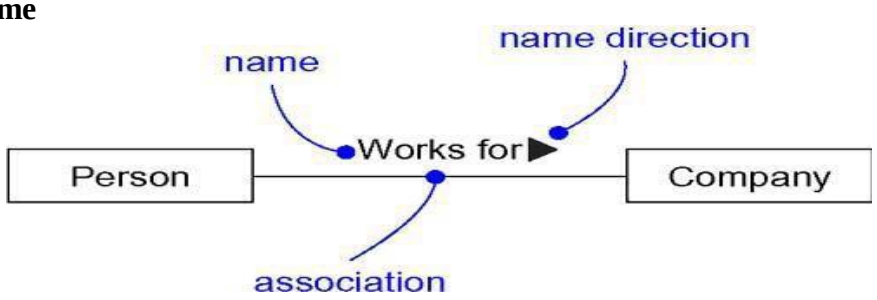


Fig: Association Names

- An association can have a name, and you use that name to describe the nature of the relationship

Role

- When a class participates in an association, it has a specific role that it plays in that relationship;
- The same class can play the same or different roles in other associations.
- An instance of an association is called a link

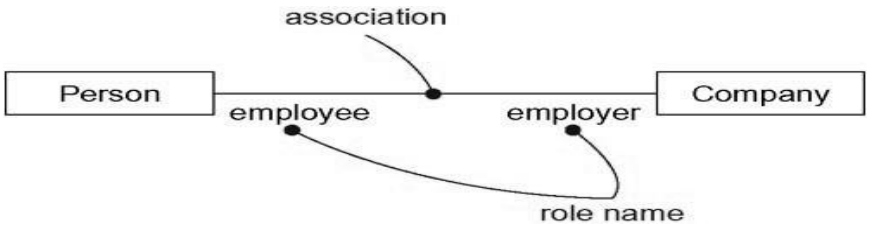


Fig: Role Names

Multiplicity

- In many modeling situations, it's important for you to state how many objects may be connected across an instance of an association
- This "how many" is called the multiplicity of an association's role
- You can show a multiplicity of exactly one (1), zero or one (0..1), many (0..*), or one or more (1..*). You can even state an exact number (for example, 3).

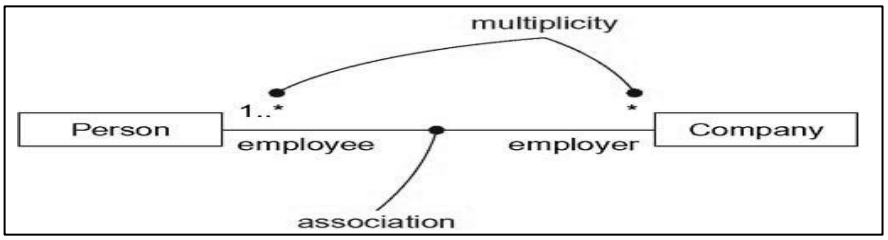


Fig: Multiplicity

Aggregation

- Sometimes, you will want to model a "whole/part" relationship, in which one class represents a larger thing (the "whole"), which consists of smaller things (the "parts").
- This kind of relationship is called aggregation, which represents a "has-a" relationship, meaning that an object of the whole has objects of the part
- Aggregation is really just a special kind of association and is specified by adorning a plain association with an open diamond at the whole end

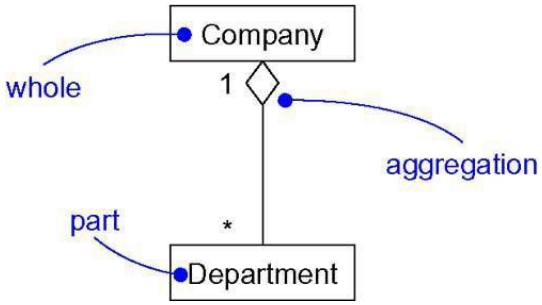


Fig: Aggregation

Common Modeling Techniques

Modeling Simple Dependencies

The most common kind of dependency relationship is the connection between classes that only uses another class as a parameter to an operation.

To model simple dependencies

- Create a dependency pointing from the class with the operation to the class used as a parameter in the operation.
- The following figure shows a set of classes drawn from a system that manages the assignment of students and instructors to courses in a university.
- This figure shows a dependency from CourseSchedule to Course, because Course is used in both the add and remove operations of CourseSchedule.
- The dependency from Iterator shows that the Iterator uses the CourseSchedule; the CourseSchedule knows nothing about the Iterator. The dependency is marked with a stereotype, which specifies that this is not a plain dependency, but, rather, it represents a friend, as in C++.

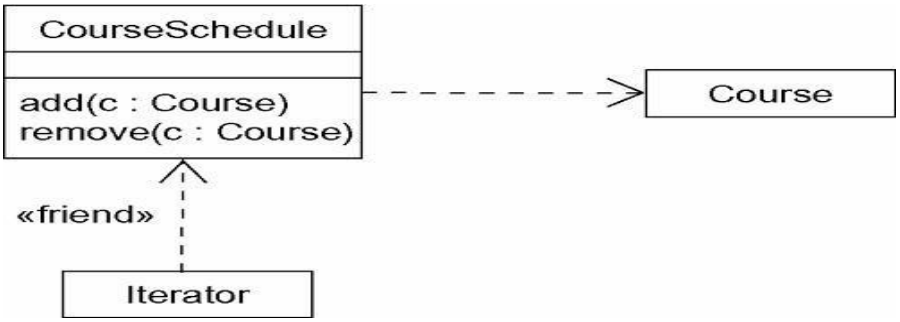


Fig: Modeling simple dependency relationships

Modeling Single Inheritance

To model single inheritance relationships

- Given a set of classes, look for responsibilities, attributes, and operations that are common to two or more classes.
- Elevate these common responsibilities, attributes, and operations to a more general class. If necessary, create a new class to which you can assign these
- Specify that the more-specific classes inherit from the more-general class by placing a generalization relationship that is drawn from each specialized class to its more-general parent.

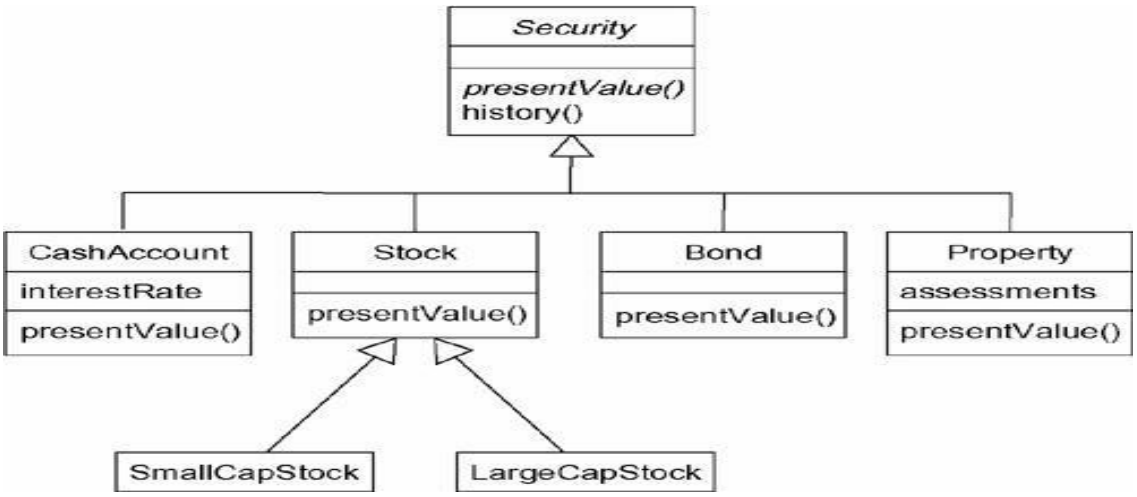


Fig: Inheritance Relationships

Modeling Structural Relationships

- When you model with dependencies or generalization relationships, you are modeling classes that represent different levels of importance or different levels of abstraction
- Given a generalization relationship between two classes, the child inherits from its parent but the parent has no specific knowledge of its children.
- Dependency and generalization relationships are one-sided.
- Associations are, by default, bidirectional; you can limit their direction
- Given an association between two classes, both rely on the other in some way, and you can navigate in either direction

- An association specifies a structural path across which objects of the classes interact.

To model structural relationships

- For each pair of classes, if you need to navigate from objects of one to objects of another, specify an association between the two. This is a data-driven view of associations.

- For each pair of classes, if objects of one class need to interact with objects of the other class other than as parameters to an operation, specify an association between the two. This is more of a behavior-driven view of associations.

- For each of these associations, specify a multiplicity (especially when the multiplicity is not *, which is the default), as well as role names (especially if it helps to explain the model).

- If one of the classes in an association is structurally or organizationally a whole compared with the classes at the other end that look like parts, mark this as an aggregation by adorning the association at the end near the whole

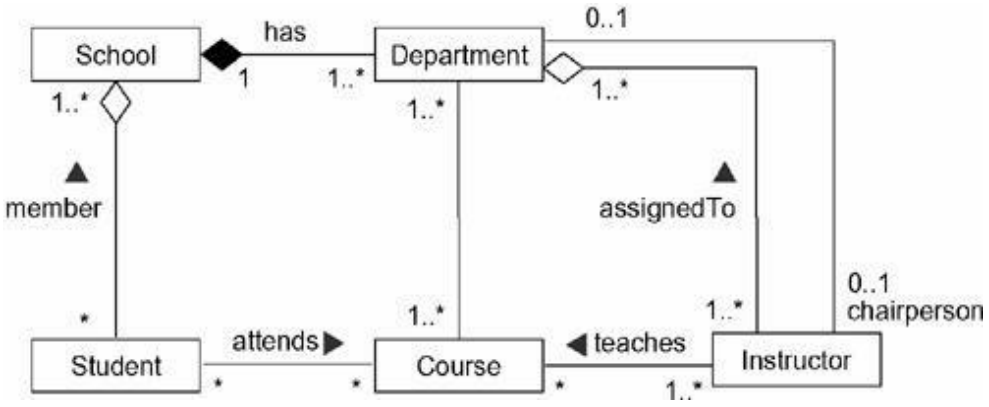


Fig: Structural Relationships

Common Mechanisms

- The UML is made simpler by the presence of four common mechanisms that apply consistently throughout the language: specifications, adornments, common divisions, and extensibility mechanisms.
- This chapter explains the use of two of these common mechanisms, adornments and extensibility mechanisms.
- Notes are the most important kind of adornment that stands alone.
- A note is a graphical symbol for rendering constraints or comments attached to an element or a collection of elements.
- You use notes to attach information to a model, such as requirements, observations, reviews, and explanations.
- The UML's extensibility mechanisms permit you to extend the language in controlled ways.
- These mechanisms include stereotypes, tagged values, and constraints.

- A stereotype extends the vocabulary of the UML, allowing you to create new kinds of building blocks that are derived from existing ones but that are specific to your problem.
- A tagged value extends the properties of a UML building block, allowing you to create new information in that element's specification.
- A constraint extends the semantics of a UML building block, allowing you to add new rules or modify existing ones. You use these mechanisms to tailor the UML to the specific needs of your domain and your development culture.

Terms and Concepts

- A **note** is a graphical symbol for rendering constraints or comments attached to an element or a collection of elements. Graphically, a note is rendered as a rectangle with a dog-eared corner, together with a textual or graphical comment.

- A **stereotype** is an extension of the vocabulary of the UML, allowing you to create new kinds of building blocks similar to existing ones but specific to your problem. Graphically, a stereotype is rendered as a name enclosed by guillemets and placed above the name of another element. As an option, the stereotyped element may be rendered by using a new icon associated with that stereotype.

- A **tagged value** is an extension of the properties of a UML element, allowing you to create new information in that element's specification. Graphically, a tagged value is rendered as a string enclosed by brackets and placed below the name of another element.

- A **constrain**t is an extension of the semantics of a UML element, allowing you to add new rules or to modify existing ones. Graphically, a constraint is rendered as a string enclosed by brackets and placed near the associated element or connected to that element or elements by dependency relationships. As an alternative, you can render a constraint in a note.

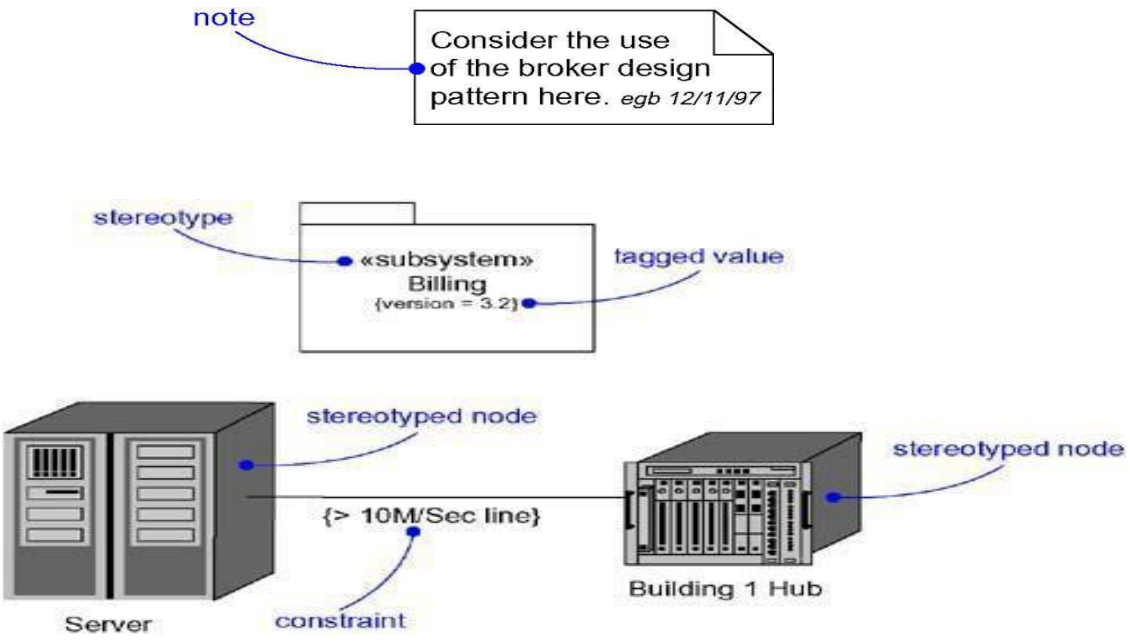


Fig: Note, Stereotypes, Tagged Values, and Constraints

Note

- A note is a graphical symbol for rendering constraints or comments attached to an element or a collection of elements
- Graphically, a note is rendered as a rectangle with a dog-eared corner, together with a textual or graphical comment.
- A note may contain any combination of text or graphics. If your implementation allows it,
- You can put a live URL inside a note, or even link to or embed another ocument. In this way, you can use the UML to organize all the artifacts you might generate or use during development.

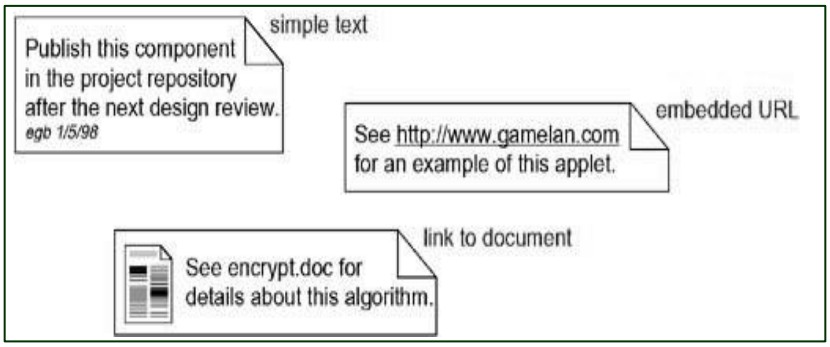


Fig: Notes

Stereotypes

- A stereotype is an extension of the vocabulary of the UML, allowing you to create new kinds of building blocks similar to existing ones but specific to your problem.
- Graphically, a stereotype is rendered as a name enclosed by guillemets and placed above the name of another element.

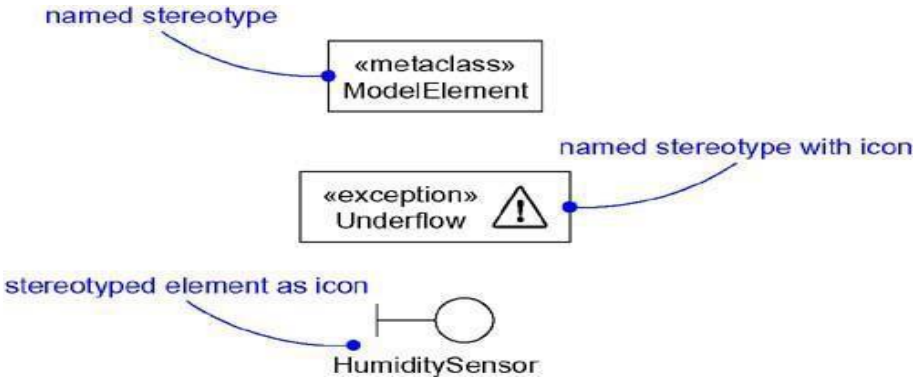


Fig: Stereotypes

Tagged Values

- Every thing in the UML has its own set of properties: classes have names, attributes, and operations; associations have names and two or more ends (each with its own properties); and so on.
- With stereotypes, you can add new things to the UML; with tagged values, you can add new properties.

- A tagged value is not the same as a class attribute. Rather, you can think of a tagged value as metadata because its value applies to the element itself, not its instances.
- A tagged value is an extension of the properties of a UML element, allowing you to create new information in that element's specification.
- Graphically, a tagged value is rendered as a string enclosed by brackets and placed below the name of another element.
- In its simplest form, a tagged value is rendered as a string enclosed by brackets and placed below the name of another element.
- That string includes a name (the tag), a separator (the symbol =), and a value (of the tag).

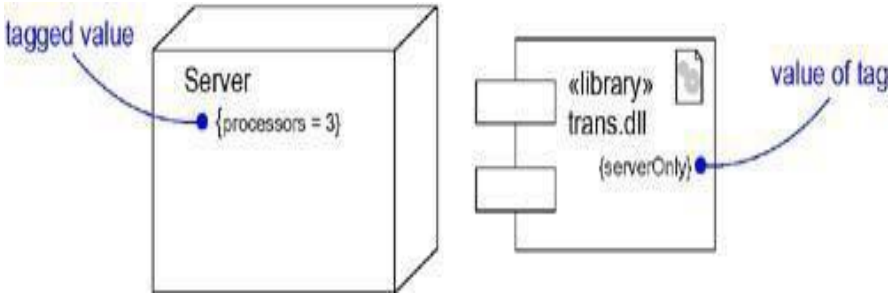


Fig: Tagged Values

Constraints

- A constraint specifies conditions that must be held true for the model to be well-formed.
- A constraint is rendered as a string enclosed by brackets and placed near the associated element
- Graphically, a constraint is rendered as a string enclosed by brackets and placed near the associated element or connected to that element or elements by dependency relationships.

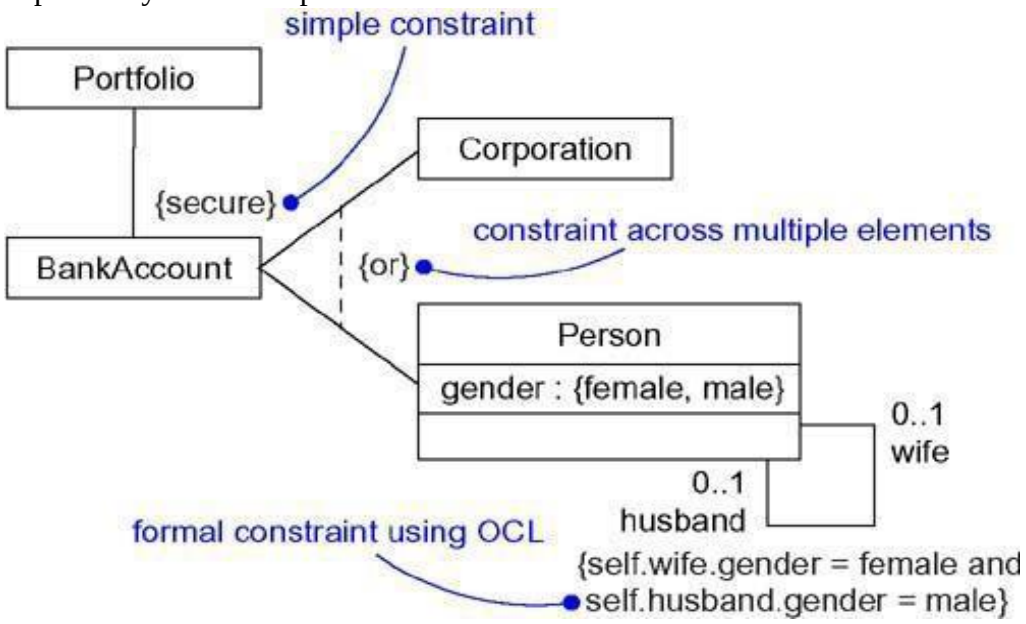


Fig: Constraints

Common Modeling Techniques

Modeling Comments

- The most common purpose for which you'll use notes is to write down free-form observations, reviews, or explanations.
- By putting these comments directly in your models, your models can become a common repository for all the disparate artifacts you'll create during development.

To model a comment

- Put your comment as text in a note and place it adjacent to the element to which it refers
- Remember that you can hide or make visible the elements of your model as you see fit.
- If your comment is lengthy or involves something richer than plain text, consider putting your comment in an external document and linking or embedding that document in a note attached to your model
- As your model evolves, keep those comments that record significant decisions that cannot be inferred from the model itself, and• unless they are of historic interest discard the others.

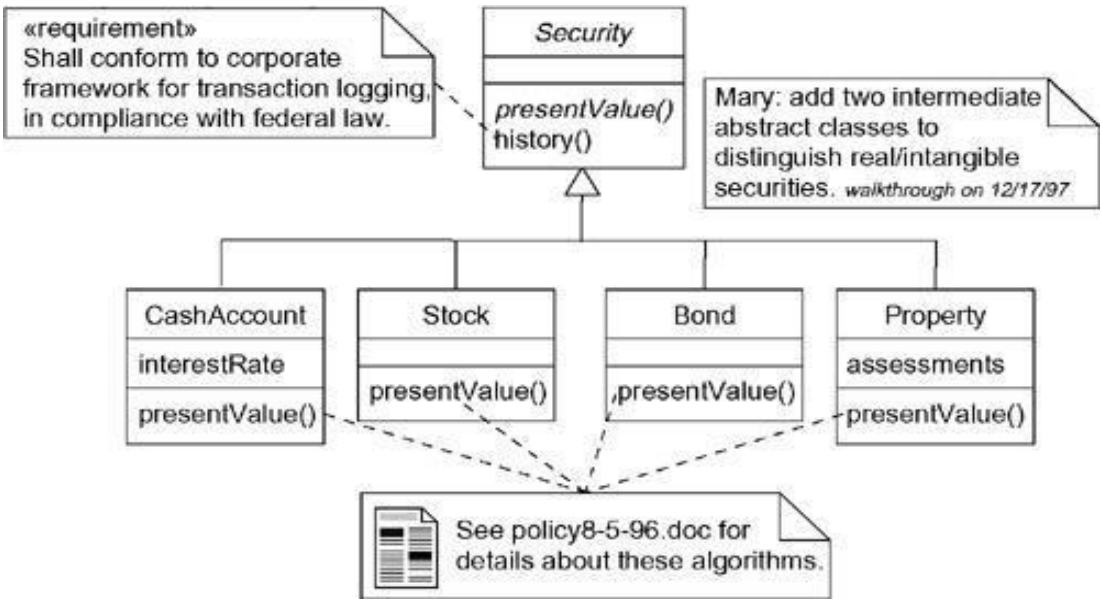


Fig: Modeling Comments

- For example, the above figure shows a model that's a work in progress of a class hierarchy, showing some requirements that shape the model, as well as some notes from a design review.
- In this example, most of the comments are simple text (such as the note to Mary), but one of them (the note at the bottom of the diagram) provides a hyperlink to another document

Modeling New Building Blocks

- The UML's building blocks—classes, interfaces, collaborations, components, nodes, associations, and so on—are generic enough to address most of the things you'll want to model.
- However, if you want to extend your modeling vocabulary or give distinctive visual cues to certain kinds of abstractions that often appear in your domain, you need to use stereotypes.

To model new building blocks

- Make sure there's not already a way to express what you want by using basic UML
- If you're convinced there's no other way to express these semantics, identify the primitive thing in the UML that's most like what you want to model and define a new stereotype for that thing
- Specify the common properties and semantics that go beyond the basic element being stereotyped by defining a set of tagged values and constraints for the stereotype.
- If you want these stereotype elements to have a distinctive visual cue, define a new icon for the stereotype

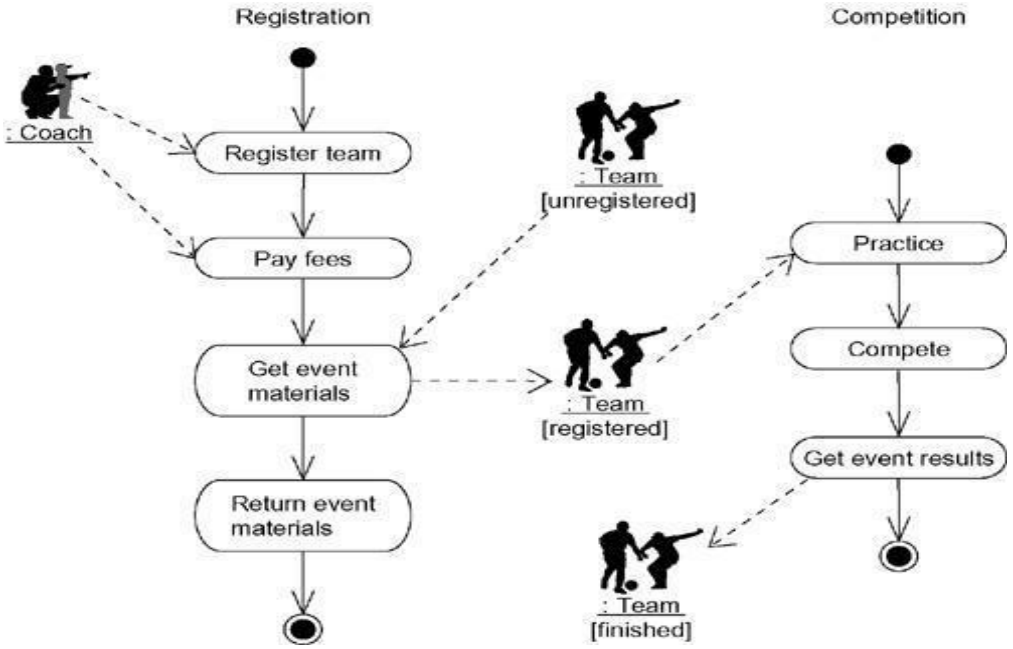


Fig: Modeling New Building Blocks.

- The above Figure shows, there are two things that stand out-Coach objects and Team objects. These are not just plain kinds of classes. Rather, they are now primitive building blocks that you can use in this context.
- You can create these new building blocks by defining a coach and team stereotype and applying them to UML's classes. In this figure, the anonymous instances called: Coach and: Team (the latter shown in various states namely, unregistered, registered, and finished) appear using the icons associated with these stereotypes.

Modeling New Properties

- The basic properties of the UML's building blocks—attributes and operations for classes, the contents of packages, and so on—are generic enough to address most of the things you'll want to model.
- However, if you want to extend the properties of these basic building blocks, you need to use tagged values.

To model new properties

- First, make sure there's not already a way to express what you want by using basic UML

- If you're convinced there's no other way to express these semantics, add this new property to an individual element or a stereotype.

- Figure shows four subsystems, each of which has been extended to include its version number and status. In the case of the Billing subsystem, one other tagged value is shown the person who has currently checked out the subsystem.

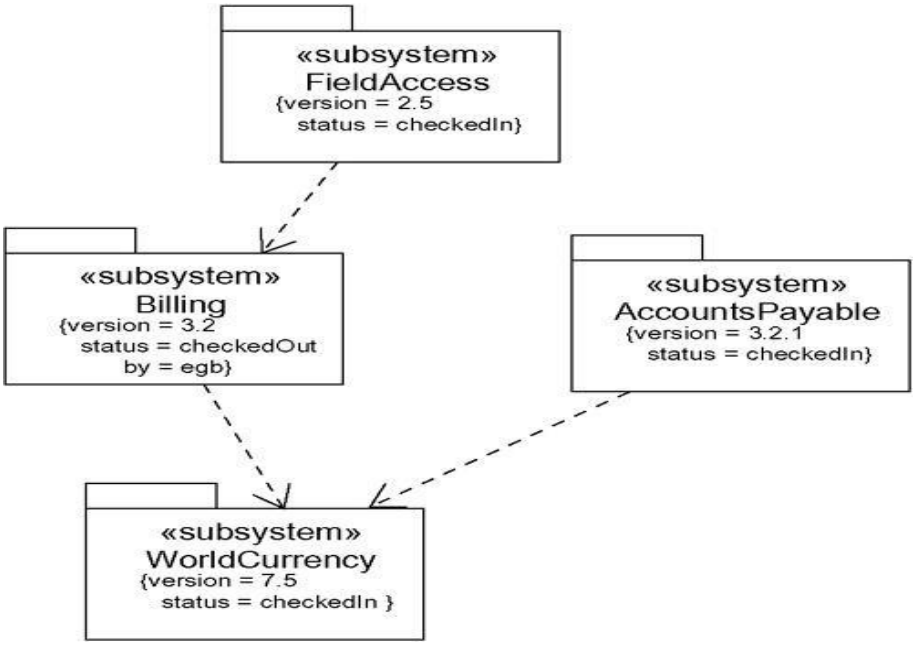


Fig: Modeling New Properties

Modeling New Semantics

- When you create a model using the UML, you work within the rules the UML lays down
- However, if you find yourself needing to express new semantics about which the UML is silent or that you need to modify the UML's rules, then you need to write a constraint.

To model new semantics

- First, make sure there's not already a way to express what you want by using basic UML
- If you're convinced there's no other way to express these semantics, write your new semantics as text in a constraint and place it adjacent to the element to which it refers

- If you need to specify your semantics more precisely and formally, write your new semantics using OCL.

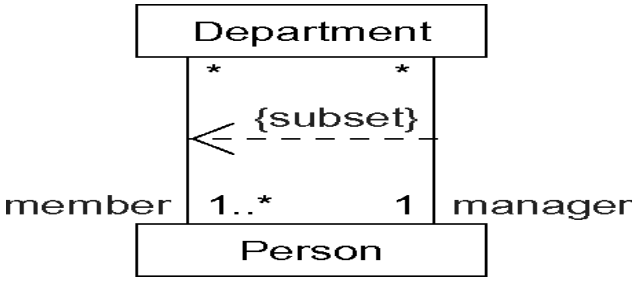


Fig: Modeling New Semantics

- This diagram shows that each Person may be a member of zero or more Departments and that each Department must have at least one Person as a member.
- This diagram goes on to indicate that each Department must have exactly one Person as a manager and every Person may be the manager of zero or more Departments. All of these semantics can be expressed using simple UML.

Diagrams

- When you view a software system from any perspective using the UML, you use diagrams to organize the elements of interest.
- The UML defines nine kinds of diagrams, which you can mix and match to assemble each view.
- Of course, you are not limited to these nine diagrams. In the UML, these nine are defined because they represent the most common packaging of viewed elements. To fit the needs of your project or organization, you can create your own kinds of diagrams to view UML elements in different ways.
- You'll use the UML's diagrams in two basic ways:
    - To specify models from which you'll construct an executable system (forward engineering)
    - To reconstruct models from parts of an executable system (reverse engineering).

Terms and Concepts

System

- A system is a collection of subsystems organized to accomplish a purpose and described by a set of models, possibly from different viewpoints

Subsystem

- A subsystem is a grouping of elements, of which some constitute a specification of the behavior offered by the other contained elements.

Model

- A model is a semantically closed abstraction of a system, meaning that it represents a complete and self-consistent simplification of reality, created in order to better understand the system. In the context of architecture

View

- view is a projection into the organization and structure of a system's model, focused on one aspect of that system

Diagram

- A diagram is the graphical presentation of a set of elements, most often rendered as a connected graph of vertices (things) and arcs (relationships).
- A diagram is just a graphical projection into the elements that make up a system
- Each diagram provides a view into the elements that make up the system
- Typically, you'll view the static parts of a system using one of the four following diagrams.
  - Class diagram
  - Object diagram
  - Component diagram
  - Deployment diagram

You'll often use five additional diagrams to view the dynamic parts of a system.

- Use case diagram
- Sequence diagram
- Collaboration diagram
- Statechart diagram
- Activity diagram.

The UML defines these nine kinds of diagrams.

- Every diagram you create will most likely be one of these nine or occasionally of another kind
- Every diagram must have a name that's unique in its context so that you can refer to a specific diagram and distinguish one from another
- You can project any combination of elements in the UML in the same diagram. For example, you might show both classes and objects in the same diagram

Structural Diagrams

- The UML's four structural diagrams exist to visualize, specify, construct, and document the static aspects of a system.
- The UML's structural diagrams are roughly organized around the major groups of things you'll find when modeling a system.
  - Class diagram : Classes, interfaces, and collaborations
  - Object diagram : Objects
  - Component diagram : Components
  - Deployment diagram : Nodes

Class Diagram

- We use class diagrams to illustrate the static design view of a system.
- Class diagrams are the most common diagram found in modeling object-oriented systems.
- A class diagram shows a set of classes, interfaces, and collaborations and their relationships.
- Class diagrams that include active classes are used to address the static process view of a system.

Object Diagram

- Object diagrams address the static design view or static process view of a system just as do class diagrams, but from the perspective of real or prototypical cases.
- An object diagram shows a set of objects and their relationships.
- You use object diagrams to illustrate data structures, the static snapshots of instances of the things found in class diagrams.

Component Diagram

- We use component diagrams to illustrate the static implementation view of a system.
- A component diagram shows a set of components and their relationships.
- Component diagrams are related to class diagrams in that a component typically maps to one or more classes, interfaces, or collaborations.

Deployment Diagram

- We use deployment diagrams to illustrate the static deployment view of an architecture.
- A deployment diagram shows a set of nodes and their relationships.
- Deployment diagrams are related to component diagrams in that a node typically encloses one or more components.

Behavioral Diagrams

- The UML's five behavioral diagrams are used to visualize, specify, construct, and document the dynamic aspects of a system.
- The UML's behavioral diagrams are roughly organized around the major ways you can model the dynamics of a system.

- Use case diagram : Organizes the behaviors of the system
- Sequence diagram : Focused on the time ordering of messages
- Collaboration diagram : Focused on the structural organization of objects that Send and receive messages
- Statechart diagram : Focused on the changing state of a system driven by Events
- Activity diagram : Focused on the flow of control from activity to Activity

Use Case Diagram

- A use case diagram shows a set of use cases and actors and their relationships.
- We apply use case diagrams to illustrate the static use case view of a system.
- Use case diagrams are especially important in organizing and modeling the behaviors of a system.

Sequence Diagram

- We use sequence diagrams to illustrate the dynamic view of a system.
- A sequence diagram is an interaction diagram that emphasizes the time ordering of messages.
- A sequence diagram shows a set of objects and the messages sent and received by those objects.
- The objects are typically named or anonymous instances of classes, but may also represent instances of other things, such as collaborations, components, and nodes.

Collaboration Diagram

- We use collaboration diagrams to illustrate the dynamic view of a system.

- A collaboration diagram is an interaction diagram that emphasizes the structural organization of the objects that send and receive messages.
- A collaboration diagram shows a set of objects, links among those objects, and messages sent and received by those objects.
- The objects are typically named or anonymous instances of classes, but may also represent instances of other things, such as collaborations, components, and nodes.

***** Sequence and collaboration diagrams are isomorphic, meaning that you can convert from one to the other without loss of information.

Statechart Diagram

- We use statechart diagrams to illustrate the dynamic view of a system.
- They are especially important in modeling the behavior of an interface, class, or collaboration.
- A statechart diagram shows a state machine, consisting of states, transitions, events, and activities.
- Statechart diagrams emphasize the event-ordered behavior of an object, which is especially useful in modeling reactive systems.

Activity Diagram

- We use activity diagrams to illustrate the dynamic view of a system.
- Activity diagrams are especially important in modeling the function of a system.
- Activity diagrams emphasize the flow of control among objects.
- An activity diagram shows the flow from activity to activity within a system.
- An activity shows a set of activities, the sequential or branching flow from activity to activity, and objects that act and are acted upon.

Common Modeling Techniques

Modeling Different Views of a System

- When you model a system from different views, you are in effect constructing your system simultaneously from multiple dimensions.

To model a system from different views

- Decide which views you need to best express the architecture of your system and to expose the technical risks to your project
- For each of these views, decide which artifacts you need to create to capture the essential details of that view.
- As part of your process planning, decide which of these diagrams you'll want to put under some sort of formal or semi-formal control.

- For example, if you are modeling a simple monolithic application that runs on a single machine, you might need only the following handful of diagrams.

• Use case view	:	Use case diagrams
• Design view	:	Class diagrams (for structural modeling) Interaction diagrams (for behavioral modeling)
• Process view	:	None required
• Implementation view	:	None required
• Deployment view	:	None required

- If yours is a reactive system or if it focuses on process flow, you'll probably want to include statechart diagrams and activity diagrams, respectively, to model your system's behavior.

- Similarly, if yours is a client/server system, you'll probably want to include component diagrams and deployment diagrams to model the physical details of your system.

- Finally, if you are modeling a complex, distributed system, you'll need to employ the full range of the UML's diagrams in order to express the architecture of your system and the technical risks to your project, as in the following.

• Use case view	:	Use case diagrams Activity diagrams (for behavioral modeling)
• Design view	:	* Class diagrams (for structural modeling) * Interaction diagrams (for behavioral modeling) * Statechart diagrams (for behavioral modeling)
• Process view	:	* Class diagrams (for structural modeling) * Interaction diagrams (for behavioral modeling)
• Implementation view	:	Component diagram
• Deployment view	:	Deployment diagrams

Modeling Different Levels of Abstraction

- Not only do you need to view a system from several angles, you'll also find people involved in development who need the same view of the system but at different levels of abstraction
- Basically, there are two ways to model a system at different levels of abstraction:
    1. By presenting diagrams with different levels of detail against the same model
    2. By creating models at different levels of abstraction with diagrams that trace from one model to another.

To model a system at different levels of abstraction by presenting diagrams with different levels of detail,

- Consider the needs of your readers, and start with a given model
- If your reader is using the model to construct an implementation, she'll need diagrams that are at a lower level of abstraction which means that they'll need to reveal a lot of detail
- If she is using the model to present a conceptual model to an end user, she'll need diagrams that are at a higher level of abstraction which means that they'll hide a lot of detail
- Depending on where you land in this spectrum of low-to-high levels of abstraction, create a diagram at the right level of abstraction by hiding or revealing the following four categories of things from your model:

Building blocks and relationships:

- Hide those that are not relevant to the intent of your diagram or the needs of your reader.

Adornments:

- Reveal only the adornments of these building blocks and relationships that are essential to understanding your intent.

Flow:

- In the context of behavioral diagrams, expand only those messages or transitions that are essential to understanding your intent.

Stereotypes:

- In the context of stereotypes used to classify lists of things, such as attributes and operations, reveal only those stereotyped items that are

essential to understanding your intent.

- The **main advantage** of this approach is that you are always modeling from a common semantic repository.

- The **main disadvantage** of this approach is that changes from diagrams at one level of abstraction may make obsolete diagrams at a different level of abstraction.

To model a system at different levels of abstraction by creating models at different levels of abstraction,

- Consider the needs of your readers and decide on the level of abstraction that each should view, forming a separate model for each level.
- In general, populate your models that are at a high level of abstraction with simple abstractions and your models that are at a low level of abstraction with detailed abstractions. Establish trace dependencies among the related elements of different models.
- In practice, if you follow the five views of an architecture, there are four common situations you'll encounter when modeling a system at different levels of abstraction:

Use cases and their realization:
Use cases in a use case model will trace to collaborations in a design model.

Collaborations and their realization:
Collaborations will trace to a society of classes that work together to carry out the collaboration.

Components and their design:
Components in an implementation model will trace to the elements in a design model.

Nodes and their components:
Nodes in a deployment model will trace to components in an implementation model. The **main advantage** of the approach is that diagrams at different levels of abstraction remain more loosely coupled. This means that changes in one model will have less direct effect on other models.

The **main disadvantage** of this approach is that you must spend resources to keep these models and their diagrams synchronized

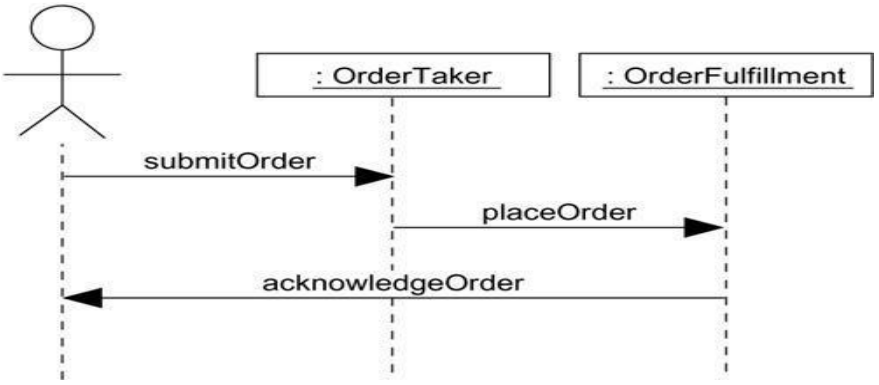


Fig: Interaction Diagram at a High Level of Abstraction

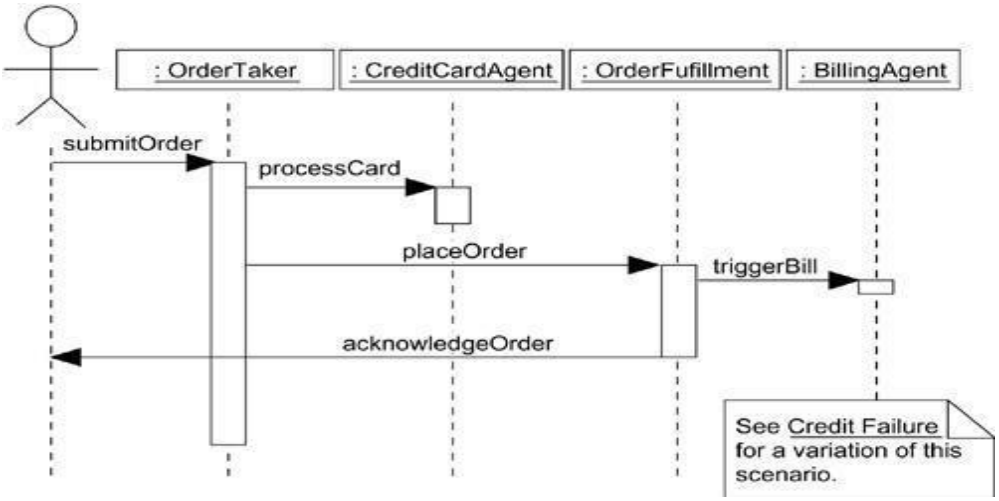


Fig: Interaction at a Low Level of Abstraction

* Both of these diagrams work against the same model, but at different levels of detail.

Modeling Complex Views

To model complex views

- First, convince yourself there's no meaningful way to present this information at a higher level of abstraction, perhaps eliding some parts of the diagram and retaining the detail in other parts.

- If you've hidden as much detail as you can and your diagram is still complex, consider grouping some of the elements in packages or in higher level collaborations, then render only those packages or collaborations in your diagram.

- If your diagram is still complex, use notes and color as visual cues to draw the reader's attention to the points you want to make.

- If your diagram is still complex, print it in its entirety and hang it on a convenient large wall. You lose the interactivity an online version of the diagram brings, but you can step back from the diagram and study it for common patterns.